

Algoritmos e Estruturas de Dados II

Corretude de algoritmos iterativos e invariantes de laço — Parte 1

Prof. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe

`andreyoshiaki@dcomp.ufs.br`

Por que testar um algoritmo não basta para garantir que ele é correto



O que é um **invariante de laço** e as três propriedades que o tornam uma prova



Prova de corretude da **busca do máximo** em um vetor



Prova de corretude da **ordenação por inserção**

Corretude e invariantes de laço

Primeiro algoritmo: busca do máximo

Segundo algoritmo: ordenação por inserção

Corretude e invariantes de laço

Problema:

- Testes mostram que o algoritmo funciona *nos casos testados* — nunca em todos.
- Um vetor de 20 inteiros de 32 bits tem mais entradas possíveis do que átomos observáveis conseguiríamos testar.
- Laços são a principal fonte de erros sutis: uma iteração a mais, uma a menos, um índice trocado.

Solução:

- Um *argumento matemático* que vale para **toda** entrada válida.
- Para algoritmos iterativos, a ferramenta central é o **invariante de laço**.
- A mesma ideia que usamos para projetar o laço serve para *provar* que ele funciona.

Definição (corretude)

Um algoritmo é **correto** quando, para *toda* entrada que satisfaz a **precondição**, ele **termina** e sua saída satisfaz a **poscondição**.

Exemplo: busca do máximo

Precondição: vetor $A[0..n-1]$ com $n \geq 1$ elementos.

Poscondição: o valor devolvido é $\max\{A[0], A[1], \dots, A[n-1]\}$.

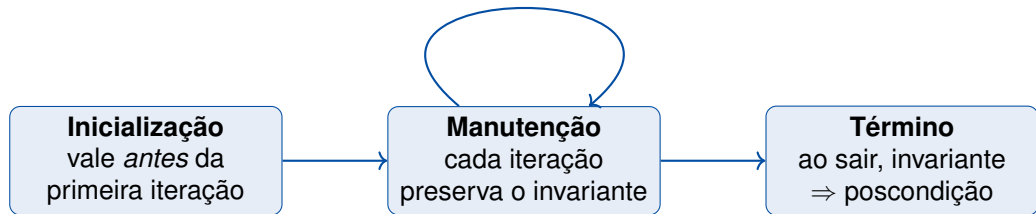
- Corretude *parcial*: **se** terminar, a resposta é certa.
- Corretude *total*: parcial + garantia de **término**.

Definição

Um **invariante de laço** é uma afirmação sobre as variáveis do programa que é **verdadeira no início de cada iteração** do laço.

Intuição

É a “promessa” que o laço mantém enquanto trabalha: descreve *o que já foi conquistado* até aquele ponto. Ao final, a promessa acumulada deve implicar o resultado desejado.



- É o esquema da **indução matemática**: inicialização é o caso base, manutenção é o passo indutivo — e a “indução” para quando o laço termina.
- Para corretude *total*, mostramos também que o laço **termina** (ex.: uma quantidade inteira não negativa que decresce a cada iteração).

O que aprendemos

- Testar mostra presença de acertos, não ausência de erros: corretude exige prova.
- Corretude = precondição $\rightarrow$ poscondição + término.
- Invariante de laço: afirmação verdadeira no início de toda iteração.
- Prova em três passos: inicialização, manutenção e término.

Primeiro algoritmo: busca do máximo

Problema:

Dado um vetor $A[0..n-1]$ com $n \geq 1$, encontrar o valor máximo.

- Algoritmo de uma linha de raciocínio — ideal para ver a *mecânica* da prova sem distrações.

Solução:

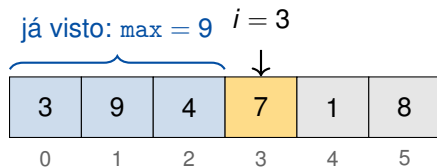
Percorrer o vetor guardando o maior valor *já visto*.

- “O maior já visto” é exatamente o invariante — a frase informal é a afirmação formal.

```
1  int maximo(int A[], int n) {    /* pre: n >= 1          */
2      int max = A[0];
3      for (int i = 1; i < n; i++) {
4          /* invariante: max == maior valor de A[0..i-1] */
5          if (A[i] > max)
6              max = A[i];
7      }
8      return max;                /* pos: max(A[0..n-1]) */
9  }
```

Invariante

No início da iteração de índice i : $\text{max} = \max\{A[0], \dots, A[i-1]\}$.



- Azul: prefixo $A[0..i - 1]$ já examinado — o invariante fala **só** sobre ele.
- Amarelo: elemento da iteração atual; cinza: ainda não examinado.
- A iteração “incorpora” $A[i]$ ao prefixo e restaura o invariante para $i + 1$.

- **Inicialização** ($i = 1$): $\max = A[0] = \max\{A[0]\}$ — o invariante vale para o prefixo de tamanho 1. ✓
- **Manutenção**: suponha $\max = \max\{A[0..i-1]\}$ no início da iteração i .
 - Se $A[i] > \max$: novo $\max = A[i] = \max\{A[0..i]\}$.
 - Senão: $A[i] \leq \max$, logo $\max$ já é $\max\{A[0..i]\}$.Nos dois casos o invariante vale para $i + 1$. ✓
- **Término**: o laço para com $i = n$; o invariante dá $\max = \max\{A[0..n-1]\}$ — exatamente a poscondição. ✓
- **O laço termina?** $n - i$ é inteiro, não negativo e decresce 1 por iteração. ✓

Versão com erro: e se o laço começasse com $\text{max} = 0$?

Com $\text{max} = 0$ e $i = 0$, a “inicialização” exigiria $0 = \max\{\}$ — o prefixo vazio não tem máximo, e para o vetor $A = [-5, -2, -9]$ a função devolveria 0, que nem está no vetor.

- O passo de **inicialização** falha: a prova não sai do lugar — e o erro aparece *antes* de qualquer teste.
- Erro clássico de quem inicializa acumuladores “no chute”: o invariante obriga a escolher $\text{max} = A[0]$ (e $i = 1$).

O que aprendemos

- O invariante formaliza a frase “ max é o maior já visto”.
- Os três passos da prova foram diretos: caso base no prefixo de tamanho 1, análise de dois casos na manutenção, poscondição no término.
- A prova diagnostica erros de inicialização que testes podem não revelar.

Segundo algoritmo: ordenação por inserção

Problema:

Ordenar $A[0..n - 1]$ em ordem não decrescente.

- Agora o invariante não fala de *um valor*, mas de uma *propriedade estrutural* de um trecho do vetor.
- Há dois laços aninhados — um invariante para cada.

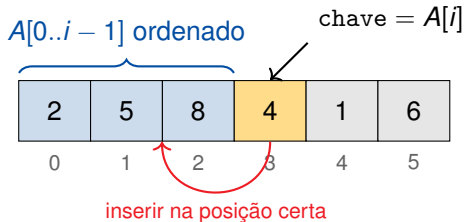
Solução:

Como organizar cartas na mão: a cada passo, inserir a próxima carta na posição certa entre as que *já estão ordenadas*.

- “As cartas na mão estão ordenadas” é o invariante do laço externo.

```
1 void insercao(int A[], int n) {
2     for (int i = 1; i < n; i++) {
3         /* invariante: A[0..i-1] ordenado, com os      */
4         /* mesmos elementos que estavam em A[0..i-1]    */
5         int chave = A[i];
6         int j = i - 1;
7         while (j >= 0 && A[j] > chave) {
8             A[j + 1] = A[j];
9             j--;
10        }
11        A[j + 1] = chave;
12    }
13 }
```

O invariante do laço externo em figura



Invariante do laço externo

No início da iteração i : o trecho $A[0..i-1]$ está **ordenado** e contém **os mesmos elementos** que ocupavam $A[0..i-1]$ originalmente.

Um algoritmo “ordenado” demais

for ($i = 0$; $i < n$; $i++$) $A[i] = i$; deixa o vetor perfeitamente ordenado — e completamente errado.

- “Estar ordenado” sozinho é uma poscondição **fraca demais**: precisamos que o resultado seja uma *permutação* da entrada.
- Por isso o invariante tem **duas** cláusulas: ordem + mesmos elementos.
- Na inserção, só *deslocamos* e *recolocamos* a chave: nenhum valor é criado ou destruído — a permutação é preservada.

- **Inicialização** ($i = 1$): $A[0..0]$ tem um único elemento — ordenado e intacto. ✓
- **Manutenção**: o laço interno desloca uma posição à direita os elementos de $A[0..i - 1]$ maiores que a chave e a insere logo antes deles.
 - Invariante do laço interno: $A[j + 2..i]$ contém os elementos deslocados, todos $>$ chave, e $A[0..j]$ está intacto.
 - Ao inserir a chave em $A[j + 1]$: $A[0..i]$ fica ordenado, com os mesmos elementos de antes. ✓
- **Término**: o laço para com $i = n$; o invariante dá $A[0..n - 1]$ ordenado e permutação da entrada — a poscondição de ordenação. ✓
- **O laço termina?** Externo: $n - i$ decresce. Interno: $j + 1 \geq 0$ decresce a cada iteração. ✓

Exemplo: rastreando o invariante em $A = [5, 2, 4, 1]$

Início de i	Vetor	Invariante: $A[0..i - 1]$ ordenado?
$i = 1$	[5 2, 4, 1]	[5] ✓
$i = 2$	[2, 5 4, 1]	[2, 5] ✓
$i = 3$	[2, 4, 5 1]	[2, 4, 5] ✓
saída ($i = 4$)	[1, 2, 4, 5]	[1, 2, 4, 5] ✓ = poscondição

- A barra separa o prefixo ordenado (invariante) do restante do vetor.
- Rastrear o invariante em um exemplo **não é a prova** — mas é o melhor jeito de *descobrir* qual invariante provar.

O que aprendemos

- Invariantes podem afirmar propriedades estruturais (“prefixo ordenado”), não apenas valores.
- A poscondição de ordenação exige duas cláusulas: ordem e permutação da entrada.
- Laços aninhados: cada laço tem seu invariante, e o do interno sustenta a manutenção do externo.

Hoje

- Invariante de laço + três passos (inicialização, manutenção, término) = prova de corretude de algoritmos iterativos.
- Busca do máximo: invariante sobre um valor acumulado.
- Ordenação por inserção: invariante estrutural com laços aninhados.

Na Parte 2

Dois algoritmos em que o invariante é menos óbvio — e mais necessário: **busca binária** (famosa por implementações erradas) e o **particionamento** do quicksort.

-  Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
Introduction to Algorithms.
MIT Press, 3 edition, 2009.
-  David Gries.
The Science of Programming.
Springer-Verlag, 1981.
-  C. A. R. Hoare.
An axiomatic basis for computer programming.
Communications of the ACM, 12(10):576–580, 1969.



Anany Levitin.

Introduction to the Design and Analysis of Algorithms.

Pearson, 3 edition, 2012.



Udi Manber.

Introduction to Algorithms: A Creative Approach.

Addison-Wesley, 1989.



Robert Sedgewick and Kevin Wayne.

Algorithms.

Addison-Wesley, 4 edition, 2011.



Nivio Ziviani.

Projeto de Algoritmos: com implementações em Pascal e C.

Cengage Learning, 3 edition, 2010.